ACPC x WISTEMS

# Women in Competitive Programming

## Workshop

ACPC x WISTEMS

WOMEN IN STEM

ACPC

WOMEN IN STEM

# What is Competitive Programming ?

ACPC x WISTEMS

ACPC

# What is Competitive Programming?

- Solving algorithmic and mathematical problems according to provided specifications in a gamified contest environment
- Time pressure, teamwork, leaderboards and prizes!
- Hopefully a fun and social activity
- Learn new things and improve your hireability!

WOMEN IN STEM

ACPC

# Choosing a Programming Language

- Our platform supports C, C++, Python and Java, which each have their pros and cons for competitive programming
- Generally, use the one you are most confident in

- **C:** A more limited form of C++, so use it only if you only know it
- **C++:** Generally our preferred language, but is pretty difficult so we won't be teaching it
- **Python:** Has some limitations compared to C++, but is more approachable
- **Java:** Some university programs focus on Java, but I would always prefer Python or C++

- We will primarily be teaching Python!

WOMEN IN STEM

ACPC

# Let’s look at some Python code!

ACPC x WISTEMS

ACPC

# Variables and Types

In Python, you can use = to create a variable. Think of it as storing some value in a named box.

Every variable roughly has a type - some type are:

- **int** - integer numbers like *1*, *0*, *-100*
- **str** - strings are text (like a group of characters): *"Hello!"*, *"123 Women in STEM!"*
- **float** - decimal numbers like *1.234*, *6.7*, etc.
- **bool** - boolean values are like yes/no answers, and are either *True* or *False*.

There are limits on what variables can be named! (e.g. cannot start with a number)

```
a = 10
b = 3.14
c = "hello"
d = False
```

# Variables and Operators

How can we manipulate the values in our variables? Operators can be used to combine different values, with different results depending on the type!

*Note: Anything after # is a comment in Python and is ignored by the code interpreter. It's just so we can leave information for us!*

```python
var1 = 1 + 2 # 3: int + int -> int
var2 = 1 / 2 # 0.5: int / int -> float
var3 = 1 == 2 # False: Are 1 and 2 the same? No they aren't
var4 = "Hello" + " " + "World" # "Hello World". We can chain these operators
x = 1
x += 1 # Sets x to be x + 1, which is 2 now
```

There are many operators! Sometimes, some type combinations don't make sense and will throw an error.
E.g. *"Hello" + 5* won't work!

# Variables and Operators

Here's some more miscellaneous operators...

```python
boolA = True
boolB = False
boolA and boolB # False. Only true if both A and B are True
boolA or boolB # True
8 // 6 # 1. This is integer division, discarding remainder
8 % 6 # 2. This is the remainder, i.e. 8 = 1*6 + 2
8 > 2 # True. 8 is greater than 2. Also >=, <, <=, !=, ==
"Hello" * 3 # "HelloHelloHello"
10**3 # 100. 10 to the power of 3
```

Please Note 10 ** 3 = 1000 not 100

# Variables and Operators

You can also convert between types. This is called *casting*!

```python
a = int("123") # 123
b = bool(0) # False
c = int(3.5) # 3. This will round towards 0
d = float(3) # 3.0
c = float("5.5") # 5.5
e = bool("False") # True. Careful! This is only False if the string is empty:
f = bool("") # False
```

This will give an error if your value doesn't make sense! (e.g. turning a string into an integer)

```python
notAnInt = int("This is clearly not an integer")
# ValueError raised! Program crashed
```

# Input and Output: I/O

In competitive programming, we need our program to be able to read in inputs, and print out outputs. There are two functions that can help with that.

**input( )**: will read a single line in as a string
**print(x)**: prints out what is inside of the bracket

```python
print(1,2,3)
# 1 2 3
# Passing multiple arguments to print space-separates them. We can customise this behaviour:
print(1,2,3,sep="\n") # "\n" means newline
# 1
# 2
# 3
myString = input() # Store a line in myString
```

# Input and Output: I/O

Now, note this input only gives us a string. Often we need to convert this into something more usable!

For instance, say we want to read a number as input and output its double.

```python
myValue = input()
print(myValue + myValue)
```

If the user inputs "1", program will output "11". This is because it reads the input as string "1", and "1"+"1" is "11" .

```python
myValue = int(input())
print(myValue + myValue)
```

Now this executes correctly, and errors if the input contains something other than an int.

# Input and Output: I/O

Another common case is a line containing space-separated integers. We can read in a list of integers as follows:

```python
myValues = list(map(int,input().split()))
```

- input().split(): Takes the line as a string and splits it at spaces: "1 2 3" turns into ["1","2","3"]
- map(int, ...) effectively produces a new list by calling int() on each element of the inputted list

# Input and Output: I/O

Sometimes we want to print out things that involve variables, you can do this using string concatenation but it isn't fun. There is a special type of string called an f-string which we can use.

```python
name = input()
print(f"Hello, {name}")
```

Notice the f before the first quote. Anything between curly braces in this string will be evaluated as Python code, so we can use it to fill in variables.

There are lots of other powerful things we can do with this to format values. One thing I'll highlight is rounding floats, say we want to print some value to 3 decimal places.

```python
a = 1.23456
print(f"{a:.3f}") # prints 1.235
```

This is a bit confusing, basically : specifies that we are going into the formatting section, .3f means we are printing a float to 3 decimal places. There are other uses you can look up.

WOMEN IN STEM

# Lists, control flow and functions

ACPC x WISTEMS

ACPC

# Lists

A list is an ordered collection of other values. Previously, our variables only stored a single value, but with lists we can store a variable amount of them!

We can construct lists using square brackets. Then, you can 'index' into them to retrieve any particular element (starting from 0).

```python
myList = ["Hello", 1, False, "this is a list"]
print(myList[0]) # prints "Hello"
print(myList[4]) # Errors! There aren't enough elements in the list
myList.append(6.7) # We can add stuff like this
print(myList[4]) # prints 6.7
myList.pop() # This will remove the last element, we can also provide an index
print(myList) # ["Hello", 1, False, "this is a list"]
```

# Lists

We can also create lists by ‘slicing’ other lists.

```python
a = [1,2,3,4]
print(a[0:3]) # [1,2,3] - note the right bound is exclusive. This is same as a[:3].
print(a[3:]) # [4]. If we omit the right bound it just takes to the end.
print(a[::2]) # [1,3]. Same as a[0:5:2]. The third number means 'step size'
print(a[-1]) # 4. We can use negative indices to get from the end
print(a[::-1]) # [4,3,2,1] step size of -1 means we are looping through backwards
```

# Lists

Here's some other things we can do to lists!

```python
a = [4,2,1,3]
print(len(a)) # 4. amount of elements in the list
a.clear() # a is now []
a = [4,2,1,3]
a.reverse() # a is now [3,1,2,4]
a.sort() # a is now [1,2,3,4]. We can sort descending order using a.sort(reversed=True)
a.count(2) # 1. How many times does 2 appear in the list?

[1,2] + [3,4] # [1,2,3,4]. Add one list to the end of another
```

Sorting will fail if we are mixing data types.
E.g. you cannot sort *[1, "Hello"]*

# Control flow: Conditionals

We can execute code only if some condition holds true. This is where these boolean (true or false) types come in handy.

```python
myInt = int(input())
if myInt == 0:
  print("The user inputted 0!")
elif myInt == 1: # This is like `else if`
  print("The user inputted 1!")
else:
  print("The user inputted something else...")
```

You can chain many elifs, and you can use types other than booleans for it. You can also do if statements inline. This prints nonempty, but it would print empty if the list was [].

```python
print("Nonempty" if [1,2,3]) else "empty")
```

# Control flow: While loops

Sometimes, we might want to repeat some code until a certain condition is true. We can use *while loops!*

```python
n = 0
while n < 100:
  print("Come to AUCPL!")
  n += 1
```

Each loop, n will increase by 1 until the loop stops. Thus, the loop will print the message 100 times.

# Control flow: For loops

Sometimes, we want to execute some code for every element in a container, such as a list.

```python
a = [1,2,3,4]
sum = 0
for x in a:
  sum += x
print(x)
```

There is also a useful range function we can try: range(n) gives 0, 1, ...., n-1 (exclusive). You can also do range(a,b) to get a, a+1, ..., b-1.

```
a = [1,2,3,4]
for i in range(len(a)):
  print(a[i])
```

ACPC

# Functions!

We have actually seen lots of functions already, such as print, range, input, int and len. They take some inputs and give us some outputs. We can define them as follows:

```python
def addTwoNumbers(a, b):
  sum = a + b
  return sum

print(a + b)
```

addTwoNumbers is the name of the function. In the parentheses is the ‘parameters’, there can be any number of these (or none, as we used *input()*).

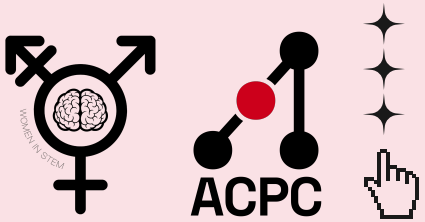
*return* is a keyword that basically gives back the value passed to it to whoever called the functions.

# Functions!

Sometimes, we want to make a function without a name, so we can just use it in one spot. There is a lambda keyword for this. For instance, lambda x: x + 1 is a function that takes an integer and returns it + 1.
This is useful in tandem with *map* as shown before, or other functions like *filter*.

```python
a = [1,2,3,4]
b = list(map(lambda x: x + 1), a)
print(b) # [2,3,4,5]
c = list(filter(lambda x: x % 2 == 0), b)
print(c) # [2,4]
```

Our second lambda is checking if x % 2 == 0, i.e. if x is even.
*filter* removes any elements that return false when given to the function passed.



# Dictionaries

Dictionaries are a way you can give a ‘key’ a ‘value’ and look up or modify the value in the future, similar to how words have definitions in a dictionary.

```
emptyDict = {} # We use curly braces to declare them
myDict = {"a": 1, "b": 2, 5: "hello"}
print(myDict["b"]) # prints 2
print(myDict[2]) # Error! No key for 2
myDict[2] = [1,2,3] # Can store whatever you want in the dict
print(myDict[2]) # [1,2,3]
del myDict["a"] # Remove a key
for key, value in myDict.items():
  print(value)
  # prints 2, "hello", [1,2,3]
```

ACPC

WOMEN IN STEM

# Let’s put it all together!

ACPC x WISTEMS

ACPC

# Some tips!

- Use your time effectively! This means not getting stuck on one problem. Maybe you could look through all the problems to gauge which ones to solve first.
- Test your code before you upload it! There is a penalty for incorrect submissions. Even if you are pretty sure it works, it's beneficial to think about edge cases and write some tests. At the very least, test the examples!

WOMEN IN STEM

ACPC

# Let’s do an example problem!

- We now know the basics of coding in Python. How do we actually competitively program?
- We have our own website https://aucpl.com that hosts the problem statements, describing the problem, how the input looks, how the output should look, and some example cases.
- The website also allows you to submit your code for grading, which is how the leaderboard works. This will run your program against a bunch of our own test cases to ensure it produces the correct output.

Here’s a problem for you to try!

- https://aucpl.com/problem/clockingchaos

WOMEN IN STEM

ACPC

# Clocking Chaos

Jack is upgrading his PC, and he has only one component left to buy, the most important of all, the CPU. Jack doesn't care about the price - all he cares about is raw processing power. Jack has been given a list of CPU specs, containing their clock speed at 1V, and the maximum voltage that a CPU can be run at. Given that a CPU's clock changes in proportion to the voltage, help Jack determine the most powerful CPU.

## Input

The first line contains a single integer $N$ ($1 \le N \le 10^6$) - the number of processors that Jack is considering.

The next line contains integers $C_1 \ldots C_N$ ($0 \le C_i \le 10^8$) - $C_i$ is the clock speed of the $i$th processor.

The final line contains decimal numbers $V_1 \ldots V_N$ ($0.0 \le V_i \le 10.0$) - $V_i$ is the maximum voltage of the $i$th processor.

## Output

Output a single number $M$ - the maximum clock speed Jack can produce, to 3 decimal places

## Example

### Input 1

```
4
3 8 2 6
1.5 2.3 5.9 1.8
```

### Output 1

```
18.400
```

$8 \times 2.3 = 18.4$

### Input 2

```
4
6 6 8 7
3.11 6.65 5.35 3.88
```

### Output 2

```
42.800
```

# Example Solution

Did you manage to solve Clocking Chaos? Here's our solution...

```python
N = int(input())
clocks = list(map(int, input().split()))
voltages = list(map(float, input().split()))
max_speed = 0
for i in range(N):
    speed = clocks[i] * voltages[i]
    if speed > max_speed:
        max_speed = speed

print(f"{max_speed:.3f}")
```

# Another Example Solution!

We can do better! At least in having less lines of code, which isn't always better (but looks cleaner!)

```python
n = int(input())

speeds = map(int, input().split())
voltages = map(float, input().split())

best_speed = max(map(lambda x: x[0] * x[1], zip(speeds, voltages)))
print(f"{best_speed:.3f}")
```

This uses some functions you may be unfamiliar with.

- Zip takes two lists and 'zips' them up together into a single list, where each element is a pair of two elements
- Max just returns the element which is the largest

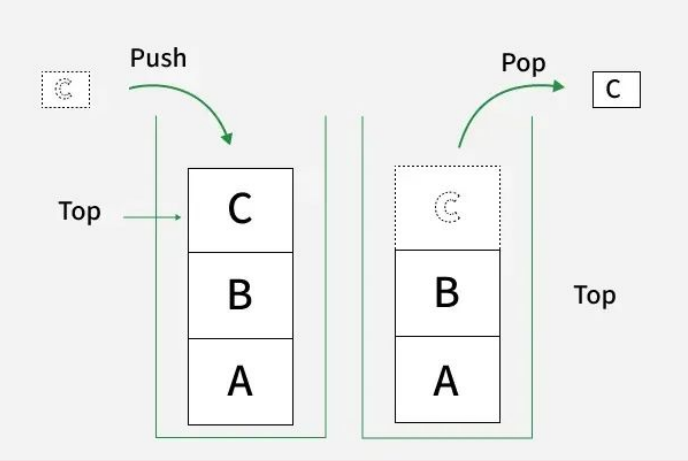
# Now, some more advanced topics!

ACPC x WISTEMS

ACPC

# Stacks

Stacks are a linear data structure that follows the LIFO (last in first out) principle. You can add elements as if it's a stack of pancakes by using 'append', and if you want to remove an element, you 'pop' it out using pop() to remove the top element.

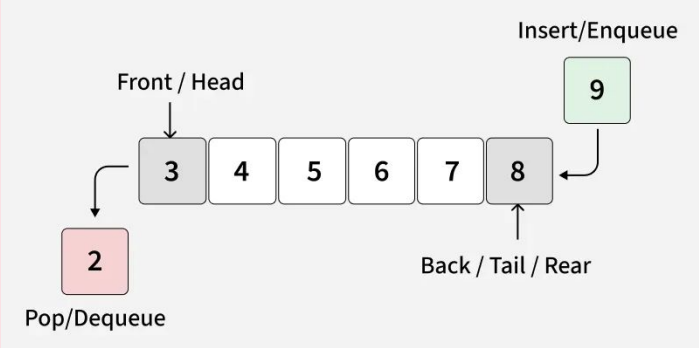



```python
# stack using list
stack = ["Amar", "Akbar", "Anthony"]
stack.append("Ram")
stack.append("Iqbal")
print(stack)

# Removes the last item
print(stack.pop())

print(stack)

# Removes the last item
print(stack.pop())

print(stack)
```

ACPC

# Queues

Queues are also a linear data structure but follow the FIFO (first in first out) principle. You can think of it like lining up to get matcha (first in first served). Again, to add a value we ‘append’ onto a queue, and to extract a value we ‘pop’ it out using pop(0).




```python
# Queue using list
queue = ["Amar", "Akbar", "Anthony"]
queue.append("Ram")
queue.append("Iqbal")
print(queue)

# Removes the first item
print(queue.pop(0))

print(queue)

# Removes the first item
print(queue.pop(0))

print(queue)
```

# Greedy Approach

This is a problem-solving strategy that builds a solution step-by-step by making the best possible choice at each moment. Ideally, these local choices will lead to the best overall solution.

- Makes the best (but irrevocable) decision in the current step
- Usually fast and efficient
- Not always optimal (picking what seems to be the best might not lead to the overall best outcome!)

Often, these approaches will involve **sorting the data**, checking if the 'best' current option **satisfies the question constraints**, and **repeat** until your solution is complete.

WOMEN IN STEM

ACPC

# Prefix Sum

A prefix sum is a running total of a list's elements. It involves iterating over a sequence of numbers and calculating the cumulative sum at each index. We can use a *for loop* to do this!

```
a = [3, 4, 1, 7, 9, 1]

res = []
cur_s= 0 # current sum

for num in a:
    cur_s += num
    res.append(cur_s)
print(res)
```

These can be useful for problems involving subarrays.

WOMEN IN STEM

ACPC

WOMEN IN STEM

# That’s all from us!

ACPC x WISTEMS

ACPC